

Documentation

Generated by docs2pdf

<https://gthlab.au/panaroo>

19 August 2026

Contents

1. Quick Start
2. Installation
3. Parameters
4. Output
5. Citation
6. Quality control
7. Add single genomes
8. Merge multiple Panaroo graphs
9. Filter presence/absence matrix
10. Pangenome association study
11. Structural variant analysis
12. Graph gene neighborhood
13. Output updated GFFs
14. Estimating the pangenome size
15. Co-Evolving Genes
16. Visualising the pangenome
17. A reference genome informed layout
18. Meta-data annotation
19. Graph correction

Quick Start

Basic usage

Using GFFs in the same format as output by Prokka run:

```
mkdir results
panaroo -i *.gff -o results --clean-mode strict
```

If you are using GFFs from RefSeq they can occasionally include annotations that do not conform the expected. This is usually due to a premature stop codon or a gene of invalid length. By default Panaroo will fail to parse these annotations. However, you can set Panaroo to ignore invalid annotation by enabling the `remove-invalid-genes` flag

```
panaroo -i *.gff -o results --clean-mode strict --remove-invalid-genes
```

Mode

By default Panaroo runs in its strictest (most conservative) mode. We have found that for most use cases this removes potential sources of contamination and error whilst retaining the majority of genes researchers are interested in.

Very rare plasmids are difficult to distinguish from contamination. Thus, if you are interested in retaining such plasmids at the expense of added contamination we recommend running panaroo using its most sensitive mode

```
panaroo -i *.gff -o results --clean-mode sensitive
```

Different input formats

Panaroo now supports multiple input formats. To use non-standard GFF3 files you must profile the input file as a list in a text file (one per line). Separate GFF and FASTA files can be provided per isolate by providing each file delimited by a space or a tab. Genbank file formats are also supported with extensions `'gbk'`, `'gb'` or `'gbff'`. These must comply with Genbank/ENA/DDJB. This can be forced in Prokka by specifying the `--compliance` parameter.

For example an input text file could look like:

```
sample_A.gff sample_A.fasta
sample_B.gbk sample_B.fasta
sample_C.gb sample_C.fasta
```

NOTE: Some annotations file such as those from RefSeq include annotations that break some of the assumptions of Panaroo. These include gene annotations of unusual length or that with premature stop codons (pseudogenes). By default Panaroo will throw an error if it encounters these annotations. You can automatically filter out such annotations by calling panaroo with the `--remove-invalid-genes` flag.

Pre-processing scripts

It is usually a good idea to perform some rudimentary quality checks on the input data prior to running Panaroo. This can be accomplished using the pre-processing script provided.

The reference mash database can be downloaded from
<https://mash.readthedocs.io/en/latest/tutorials.html>

```
panaroo-qc -t 3 --graph_type all -i *.gff --ref_db  
refseq.genomes.k21s1000.msh -o results
```

Running Prokka

Panaroo is designed to correct for many sources of error introduced during annotation. This includes refinding genes in cases where there are inconsistencies in the annotation of the same genetic sequence in different genomes.

It is also possible to force Prokka to use the same gene model for each genome. This will improve the consistency of annotations but any biases present in the model will be present in all genome annotations.

To run Prokka in this way you can build an initial model by running Prodigal on a genome assembly you believe is a reliable representative of your data set.

```
prodigal -t prodigal_training_file -c -m -g 11 -p single -q -i  
input_file.fasta > /dev/null
```

Annotations can then be generated by running Prokka with the `--prodigaltf` parameter

```
prokka --prodigaltf prodigal_training_file input_file.fasta
```

Installation

Conda is the simplest way to install Panaroo and all its dependencies.

Conda

Although Panaroo supports the latest version of python, currently Bioconda does not support versions ≥ 3.10 for all packages. An older version of python can be installed by creating an environment as

```
conda create -n panaroo python=3.9
conda activate panaroo
```

Panaroo can then be installed by running the following within a conda environment

```
conda install -c conda-forge -c bioconda -c defaults 'panaroo>=1.3'
```

alternatively it is often faster to use the [mamba](#) solver. This can be installed by running

```
conda install mamba -c conda-forge
```

Panaroo can then be installed using

```
mamba install -c conda-forge -c bioconda -c defaults 'panaroo>=1.3'
```

Manual

Install cd-hit using either conda or by following these [instructions](#).

If you would like to build multiple sequence alignments install the corresponding aligner (MAFFT by default). MAFFT can be installed using conda or by following these [instructions](#)

Alternatively cd-hit and MAFFT can be installed using apt-get on some linux systems.

```
sudo apt-get install cd-hit

sudo apt-get install mafft
```

Download or clone the repository and install by running

```
pip3 install git+https://github.com/gtonkinhill/panaroo
```

or alternatively

```
git clone https://github.com/gtonkinhill/panaroo  
cd panaroo  
python3 setup.py install
```

To install locally, instead run

```
python3 setup.py install --user
```

Panaroo relies on a number of dependencies that must be installed separately.

Dependencies

Required: * biopython * numpy * networkx * gffutils * edlib * joblib * tqdm * cd-hit

Optional: * prokka * prank * mafft * clustal * mash

Parameters

Mode

The main parameter that should be adjusted when using Panaroo is the mode. This changes a number of defaults to either favour `strict`, `moderate` or `sensitive` filtering. The default parameter settings for each of these modes is given in the table at the end of this page.

Gene Alignment

Panaroo can generate multiple sequence alignments from the resulting gene clusters. Currently, the user can choose from either [PRANK](#), [Clustal](#) or the default, [MAFFT](#). MSA can be run either for the core genome using `-a core` or for every gene cluster using `-a pan`. The frequency of a gene in the your sample required to classify it as 'core' can be set using `--core_threshold`.

Thus to align all genes present in at least 98% of isolates using clustal and 10 cpus you would run Panaroo as

```
panaroo -i *.gff -o ./results/ --clean-mode strict -a core --aligner clustal
--core_threshold 0.98 -t 10
```

You can also output unaligned gene sequences by specifying `--aligner none`. Additionally, user @revinci has provided a separate script for generating alignments after running Panaroo, which is described [here](#).

Cluster Thresholds

The Panaroo algorithm initially performs a conservative clustering step before collapsing genes into possible families. It is usually best to use the default parameters for this initial clustering stage.

Thus we recommend using the defaults for `--threshold` (0.98) and `--len_dif_percent` (0.98).

If you wish to adjust the level at which Panaroo collapses genes into putative families we suggest changing the family sequence identity level (default 0.7). Thus to run Panaroo using a more relaxed threshold of 50% identity you could run

```
panaroo -i *.gff -o ./results/ --clean-mode strict -f 0.5
```

Paralogs

Panaroo splits paralogs into separate clusters by default. Merging paralogs can be enabled by running Panaroo as

```
panaroo -i *.gff -o ./results/ --clean-mode strict --merge_paralogs
```

Refinding Genes

In order to identify genes that have been missed by annotation software, Panaroo incorporates a refinding step. Suppose two clusters geneA and geneB are adjacent in the Panaroo pangenome graph. If geneA is present in a genome but its neighbour (geneB) is not then Panaroo searches the sequence surrounding geneA for the presence of geneB. The radius of this search in nucleotides is controlled by `--search_radius`, with the default being 5000.

As such missing genes are often the results of assembly fragmentation, the refinding step only requires that a proportion of the missing gene is located. This proportion can be controlled using the `--refind_prop_match` parameter.

Thus to refind genes that match at least 50% of the sequence within a radius of 1000 nucleotides you could run

```
panaroo -i *.gff -o ./results/ --clean-mode strict --refind_prop_match 0.5 --search_radius 1000
```


Panaroo Usage

```
usage: panaroo [-h] -i INPUT_FILES [INPUT_FILES ...] -o OUTPUT_DIR
               --clean-mode {strict,moderate,sensitive}
               [--remove-invalid-genes] [-c ID] [-f FAMILY_THRESHOLD]
               [--len_dif_percent LEN_DIF_PERCENT]
               [--family_len_dif_percent FAMILY_LEN_DIF_PERCENT]
               [--merge_paralogs] [--search_radius SEARCH_RADIUS]
               [--refind_prop_match REFIN_D_PROP_MATCH]
               [--refind-mode {default,strict,off}]
               [--min_trailing_support MIN_TRAILING_SUPPORT]
               [--trailing_recursive TRAILING_RECURSIVE]
               [--edge_support_threshold EDGE_SUPPORT_THRESHOLD]
               [--length_outlier_support_proportion
LENGTH_OUTLIER_SUPPORT_PROPORTION]
               [--remove_by_consensus {True,False}]
               [--high_var_flag CYCLE_THRESHOLD_MIN]
               [--min_edge_support_sv MIN_EDGE_SUPPORT_SV]
               [--all_seq_in_graph] [--no_clean_edges] [-a {core,pan}]
               [--aligner {prank,clustal,mafft,none}] [--codons]
               [--core_threshold CORE] [--core_subset SUBSET]
               [--core_entropy_filter HC_THRESHOLD] [-t N_CPU]
               [--codon-table TABLE] [--quiet] [--version]
```

panaroo: an updated pipeline for pangenome investigation

optional arguments:

-h, --help	show this help message and exit
-t N_CPU, --threads N_CPU	number of threads to use (default=1)
--codon-table TABLE	the codon table to use for translation (default=11)
--quiet	suppress additional output
--version	show program's version number and exit

Input/output:

-i INPUT_FILES [INPUT_FILES ...], --input INPUT_FILES [INPUT_FILES ...]
input GFF3 files (usually output from running

Prokka).

Can also take a file listing each gff file line by line.

-o OUTPUT_DIR, --out_dir OUTPUT_DIR
location of an output directory

Mode:

`--clean-mode {strict,moderate,sensitive}`

The stringency mode at which to run panaroo. Must be one of 'strict', 'moderate' or 'sensitive'. Each of these modes can be fine tuned using the additional parameters in the 'Graph correction' section.

strict:

Requires fairly strong evidence (present in at least 5% of genomes) to keep likely contaminant genes. Will remove genes that are refound more often than they were called originally.

moderate:

Requires moderate evidence (present in at least 1% of genomes) to keep likely contaminant genes. Keeps genes that are refound more often than they were called originally.

sensitive:

Does not delete any genes and only performs merge and refinding operations. Useful if rare plasmids are of interest as these are often hard to disguise from contamination. Results will likely include higher number of spurious annotations.

`--remove-invalid-genes`

removes annotations that do not conform to the expected Prokka format such as those including premature stop codons.

Matching:

`-c ID, --threshold ID`

sequence identity threshold (default=0.98)

`-f FAMILY_THRESHOLD, --family_threshold FAMILY_THRESHOLD`

protein family sequence identity threshold (default=0.7)

`--len_dif_percent LEN_DIF_PERCENT`

length difference cutoff (default=0.98)

`--family_len_dif_percent FAMILY_LEN_DIF_PERCENT`

length difference cutoff at the gene family level (default=0.0)

`--merge_paralogs` don't split paralogs

Refind:

`--search_radius` SEARCH_RADIUS

the distance in nucleotides surrounding the neighbour
of an accessory gene in which to search for it

`--refind_prop_match` REFIN_PRO_P_MATCH

the proportion of an accessory gene that must be
found

in order to consider it a match

`--refind-mode` {default,strict,off}

The stringency mode at which to re-find genes.

default:

Will re-find similar gene sequences. Allows for
premature stop codons and incorrect lengths to

account

for misassemblies.

strict:

Prevents fragmented, misassembled, or potential
pseudogene sequences from being re-found.

off:

Turns off all re-finding steps.

Graph correction:

`--min_trailing_support` MIN_TRAILING_SUPPORT

minimum cluster size to keep a gene called at the end
of a contig

`--trailing_recursive` TRAILING_RECURSIVE

number of times to perform recursive trimming of low
support nodes near the end of contigs

`--edge_support_threshold` EDGE_SUPPORT_THRESHOLD

minimum support required to keep an edge that has
been

flagged as a possible mis-assembly

`--length_outlier_support_proportion` LENGTH_OUTLIER_SUPPORT_PROPORTION

proportion of genomes supporting a gene with a length
more than 1.5x outside the interquatile range for
genes in the same cluster (default=0.01). Genes
failing this test will be re-annotated at the shorter
length

`--remove_by_consensus` {True,False}

if a gene is called in the same region with similar sequence a minority of the time, remove it. One of 'True' or 'False'

`--high_var_flag CYCLE_THRESHOLD_MIN`
 minimum number of nested cycles to call a highly variable gene region (default = 5).

`--min_edge_support_sv MIN_EDGE_SUPPORT_SV`
 minimum edge support required to call structural variants in the presence/absence sv file

`--all_seq_in_graph` Retains all DNA sequence for each gene cluster in the graph output. Off by default as it uses a large amount of space.

`--no_clean_edges` Turn off edge filtering in the final output graph.

Gene alignment:

`-a {core,pan}, --alignment {core,pan}`
 Output alignments of core genes or all genes. Options are 'core' and 'pan'. Default: 'None'

`--aligner {prank,clustal,mafft,none}`
 Specify an aligner. Options: 'prank', 'clustal', and default: 'mafft'

`--codons` Generate codon alignments by aligning sequences at the protein level

`--core_threshold CORE`
 Core-genome sample threshold (default=0.95)

`--core_subset SUBSET` Randomly subset the core genome to these many genes (default=all)

`--core_entropy_filter HC_THRESHOLD`
 Manually set the Block Mapping and Gathering with Entropy (BMGE) filter. Can be between 0.0 and 1.0. By default this is set using the Tukey outlier method.

Default Parameters

\$N\$ is the number of genomes in the dataset.

Parameter	Description	strict	moderate	relaxed
min_trailing_support	Minimum number of genomes supporting a trailing cluster. Cluster with less than this will be deleted	$\max(2, \lceil 0.05 N \rceil)$	$\max(2, \lceil 0.01 N \rceil)$	2
trailing_recursive	Maximum number of times to recursively delete trailing clusters with degree ≥ 1 that have less than the minimum trailing support.	∞	∞	1
min_edge_support_sv	Minimum edge support required to be included in <code>struct_presence_absence.csv</code>	$\max(2, \lceil 0.01 N \rceil)$	$\max(2, \lceil 0.01 N \rceil)$	2
remove_by_consensus	Whether or not to remove clusters that are refound more times than they were originally annotated. This can indicate a poor initial annotation in a minority of genomes.	True	False	False
edge_support_threshold	This only affects the GML output. Edges that are below this threshold and which join clusters of higher support are removed. This improves the readability of the graph.	$\max(2, \lceil 0.01 N \rceil)$	$\max(2, \lceil 0.01 N \rceil)$	0

Output

gene_presence_absence.csv

A csv file describing which gene is in which sample. If a gene cluster is present in a sample, the sequence name of the representative for that sample is given in the matrix. The corresponding DNA and protein sequence can then be matched to those found in the `combined_DNA_CDS.fasta` and `combined_protein_CDS.fasta` files. The format is the same as that given by [Roary](#).

Annotations that have been merged will be separated by a semicolon. Refound genes that include a stop codon will have '_stop' appended to the end of the gene name. This indicates they could be a potential pseudo gene. Gene within a cluster that have an unusual length will have '_len' appended to the gene name.

gene_presence_absence.Rtab

A binary tab separated version of the `gene_presence_absence.csv` file describing which gene is in which sample. The format is the same as that given by [Roary](#).

final_graph.gml

The final pan-genome graph generated by Panaroo. This can be viewed easily using Cytoscape (see [Visualising the Pangenome](#) for more details). It includes all the meta-information such as gene annotation and which gene/edge is present in which genome. It can also be loaded into python using networkx for further processing.

struct_presence_absence.csv

A csv file which lists the presence and absence of different genomic rearrangement events. The genes involved in each event are listed in the respective column names of the csv. The thresholds for calling these events can be changed by adjusting the `--min_edge_support_sv` parameter when calling Panaroo.

pan_genome_reference.fa

This is a similar output to that produced by [Roary](#). It creates a linear reference genome of all the genes found in the dataset. The order of the genes in this reference are not significant. NOTE: to avoid issues with the multi-mapping of reads, paralogous gene clusters will only be represented once in this reference.

gene_data.csv

This is a very large file mainly used internally in the program. It links each gene sequence and annotation to the internal representations used. It can be useful in interpreting some of the output especially the 'final_graph.gml' file.

combined_DNA_CDS.fasta

This is a fasta file which includes all nucleotide sequence for both the annotated genes and those refound by the program. The gene names are the internal ones used by Panaroo. These can be translated to the original names using the 'gene_data.csv' file.

combined_protein_CDS.fasta

Similar to the combined_DNA_CDS.fasta file, this is a fasta file which includes all protein sequence for both the annotated genes and those refound by the program. The gene names are the internal ones used by Panaroo. These can be translated to the original names using the 'gene_data.csv' file.

core_gene_alignment.aln

An alignment of genes present in at least the fraction of genomes specified by the `--core_threshold` parameter (default=0.95). Currently, in cases where a gene is fragmented only the longer fragment will appear in this output.

core_gene_alignment_filtered.aln

This alignment is recommended for building core genome phylogenies. It is a filtered version of the core genome alignment. Additional genes are removed if they exceed the Block Mapping and Gathering with Entropy (BMGE) filter. This is set using the `--core_entropy_filter` parameter. By default this automatically adapts to each dataset and identifies outlying genes using Tukey's outlier test (recommended).

Citation

If you use Panaroo please cite:

Tonkin-Hill G, MacAlasdair N, Ruis C, Weimann A, Horesh G, Lees JA, Gladstone RA, Lo S, Beaudoin C, Floto RA, Frost SDW, Corander J, Bentley SD, Parkhill J. 2020. Producing polished prokaryotic pangenomes with the Panaroo pipeline. *Genome Biol* 21:180.

We include some implementations of other algorithms in the post processing scripts which should be cited separately. The citations for these scripts are given in the relevant section of the documentation.

Quality Control

Panaroo provides a pre-processing script which helps to identify possible issues with samples prior to running the full pangenome algorithm. Such checks can also be useful for many other analysis tasks.

The script can be called using 3 cpus as

```
mkdir results
panaroo-qc -i *.gff -o results -t 3 --graph_type all --ref_db
refseq.genomes.k21s1000.msh
```

This requires a reference database supplied with `--ref_db` to check for contamination. The authors of mash have provided such a database, downloadable from this [link](#).

Parameters

Generate quality control plots prior to a Panaroo run

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>-t N_CPU, --threads N_CPU</code>	number of threads to use (default=1)
<code>--graph_type {contam,ngenes,all,mds,ncontigs}</code>	the type of graph to generate (default='all')
<code>--ref_db REF_DB</code>	reference mash database for contamination quantification.
<code>--version</code>	show program's version number and exit

Input/output:

<code>-i INPUT_FILES [INPUT_FILES ...], --input INPUT_FILES [INPUT_FILES ...]</code>	input GFF3 files (usually output from running Prokka)
<code>-o OUTPUT_DIR, --out_dir OUTPUT_DIR</code>	location of an output directory

The following section includes examples from running the algorithm on a large [Klebsiella](#) dataset.

Contamination

The script wraps the popular [Mash](#) algorithm, and produces plots indicating possible sources of contamination as well as outliers.

Multi-Dimensional Scaling (MDS)

After using Mash to create a pairwise distance matrix between each sample, the script generates an MDS plot. Samples that appear as outliers in this plot should be investigated for possible contamination, poor assembly or other problems.

Outliers

The script also produces plots which help to highlight samples with an unusual number of contigs or genes. This can indicate a problem with the sample.

Add a new genome to a Panaroo graph.

Having run Panaroo it is possible to add a single genome to the resulting pangenome graph. This can be useful when we are interested in the annotations of a new genome but do not wish to rerun the entire Panaroo algorithm. This added functionality is thanks to [Daniel Anderson](#).

As an example we initially run Panaroo on the species *Klebsiella quasipneumoniae* (KpII) dataset from [Holt et al., 2015](#) and query a single genome from the sister species *Klebsiella variicola* (KpIII).

```
panaroo -i kpII/*.gff -o kp_II_panaroo -t 24 --clean-mode sensitive
```

we can now add a single genome **5150_1#7.gff** to the graph by running

```
panaroo-integrate -d kp_II_panaroo/ -i kpIII/5150_1#7.gff -t 24 -o  
updated_output
```

The updated graph as well as updated gene presence absence files are available in the folder `updated_output`

Merge Panaroo graphs

It is possible to merge pangenome graphs generated by independent runs of the Panaroo algorithm. This is particularly useful when dealing with very large or diverse datasets. The Panaroo algorithm assumes that a dataset is not overly diverse and thus results can be improved by running the algorithm on separate clusters of genomes independently before merging the resulting graphs.

This approach can also be used to compare the pangenomes of different species or lineages.

As an example we investigate the species *Klebsiella quasipneumoniae* (KpII) and *Klebsiella variicola* (KpIII) from [Holt et al., 2015](#). Initially we run Panaroo independently on each dataset in its most sensitive mode and require that all edges are kept in the final graph.

```
panaroo -i kpII/*.gff -o kp_II_panaroo -t 24 --clean-mode sensitive --no_clean_edges
panaroo -i kpIII/*.gff -o kp_III_panaroo -t 24 --clean-mode sensitive --no_clean_edges
```

we can now merge the resulting graphs using

```
mkdir kp_merge
panaroo-merge -d kp_II_panaroo/ kp_III_panaroo/ -o kp_merge -t 24
```

Filter presence/absence matrices

The Panaroo algorithm will flag certain clusters if they include potential pseudogenes, genes of unusual length or genes that have been fragmented in the process of assembly or annotation.

- potential pseudo genes will end with the suffix '_stop'
- genes with unusual lengths will end with the suffix '_len'
- fragmented genes will include multiple sequence IDs separated by a semicolon ';'

To filter these sequences out from the presence/absence matrices you can run:

```
panaroo-filter-pa -i ./gene_presence_absence.csv -o ./ --type pseudo,length
```

Parameters

```
usage: panaroo_filter_pa [-h] -i INPUT_FILE -o OUTPUT_DIR [--type TYPE]
                        [--version]
```

Filters the Panaroo gene_presence_absence.csv file to exclude sequences classed as length outliers, pseudo genes or fragmented.

optional arguments:

-h, --help	show this help message and exit
--type TYPE	any combination of 'pseudo', 'length' or 'frag' separated by commas
--version	show program's version number and exit

Input/output:

-i INPUT_FILE, --input INPUT_FILE	input 'gene_presence_absence.csv' file generated by Panaroo
-o OUTPUT_DIR, --out_dir OUTPUT_DIR	location of an output directory

Output

The program will produce new versions of the gene_presence_absence.csv and `gene_presence_absence.Rtab files with an additional suffix to indicate which kind of filtering has been performed.

Pangenome Association Study

Panaroo produces output that can be provided to [Pyseer](#) to perform pangenome association analyses. To demonstrate this we provide an example analysis of antibiotic resistance in *Neisseria gonorrhoeae*.

We look at the association between gene presence/absence and antibiotic resistance in a large [European sample](#) of 1054 isolates. Data required to replicate the analysis is downloadable at [Pathogenwatch \(eurogasp2013\)](#).

Annotation

The assemblies from the study were first annotated using Prokka

```
mkdir prokka_output
for fasta in ./genomes/*.fasta
do
  prefix=$(basename ${fasta} .fasta)
  prokka --cpus 5 --genus Neisseria --usegenus --outdir
  ./prokka_output/${prefix} --prefix $prefix $fasta
done
```

Panaroo

We can now run Panaroo, setting the option to build a multiple sequence alignment of the core genes using MAFFT.

```
mkdir panaroo_output
panaroo -i ./prokka_output/*/*.gff -o panaroo_output -t 8 --verbose -a core
```

In order to control for population structure in our association analyses we build a phylogeny from the core gene alignment using [IQ-TREE](#).

```
cd panaroo_output
iqtree -s core_gene_alignment.aln -pre core_tree -nt 8 -fast -m GTR
cd ..
```

Pyseer

We are now ready to run [Pyseer](#). We first look at association between antibiotic resistance and gene presence/absence.

```
python ~/pyseer/scripts/phylogeny_distance.py --lmm
./panaroo_output/core_tree.treefile > pyseer_out/phylogeny_K.tsv

for anti in AZM      CRO      CFM      CIP      PEN      SMX      TET
do
python ~/pyseer/pyseer-runner.py --lmm --phenotypes ./metadata/pathogenwatch-
neigo-eurogasp2013-amr-profile-1.tab --pres
./panaroo_output/gene_presence_absence.Rtab --similarity
./pyseer_out/phylogeny_K.tsv --phenotype-column $anti --output-patterns
./pyseer_out/gene_patterns_${anti}.txt > ./pyseer_out/${anti}_gwas.txt
done
```

We also have to count the number of patterns in order to control for multiple testing.

```
python ~/pyseer/scripts/count_patterns.py ./pyseer_out/gene_patterns_AZM.txt
```

This results in:

```
Patterns:      529
Threshold:     9.45E-05
```

We now have everything we need to start looking at some results! We make use of a small bit of *R* code here to summarise the results.


```
antibiotics <- c('AZM', 'CRO', 'CFM', 'CIP', 'PEN', 'SMX', 'TET')

gono_gwas <- do.call(rbind, map(antibiotics, function(ant){
  tbl <- fread(paste(c("./gono_harris_gwas/",ant,"_gwas.txt"), collapse =
""), data.table = FALSE)
  tbl$antibiotic <- ant
  return(tbl)
}))

gono_gwas <- gono_gwas[order(gono_gwas$lrt-pvalue`),]
gono_gwas <- gono_gwas[!grepl("bad-chisq", gono_gwas$notes),]

# threshold from running count_patterns in pyseer
sig_threshold <- 0.05/(529*length(antibiotics))

sum(gono_gwas$lrt-pvalue`<sig_threshold)
sig_hits <- gono_gwas[gono_gwas$lrt-pvalue`<sig_threshold,]

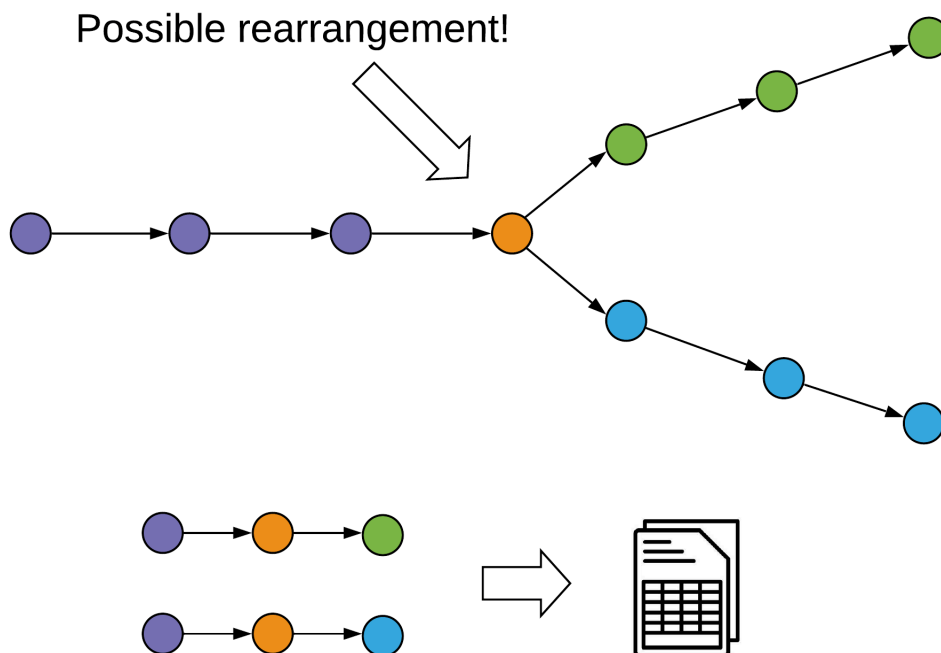
write.csv(sig_hits, file = "pan_was_results.csv", quote = FALSE, row.names =
FALSE)
```

Structural Variant Analysis

As Panaroo builds a full representation of each gene in the pangenome we can use this to investigate gene rearrangements.

To simplify association analyses between these rearrangements and phenotypes of interest we concentrate on gene triplets. As paralogs are split in the pangenome graph each sample can only be present in one path. Thus each genome can only contain one version of triplet with the same central gene. This is illustrated in the figure below.

We record the presence and absence of these gene triplets in each sample in the `struct_presence_absence.csv` file. This can be used as input to [Pyseer](#) in a similar way to the gene presence/absence matrix.



Example

We continue on from the example given in the [Gene Association Study) section.

```

for anti in AZM      CRO      CFM      CIP      PEN      SMX      TET
do
python ~/pyseer/pyseer-runner.py --lmm --phenotypes ./metadata/pathogenwatch-
neigo-eurogasp2013-amr-profile-1.tab --pres
./panaroo_output/struct_presence_absence.Rtab --similarity
./pyseer_out/phylogeny_K.tsv --phenotype-column $anti --output-patterns
./pyseer_out/struct_patterns_${anti}.txt >
./pyseer_out/${anti}_struct_was.txt
done

```

again counting the number of patterns for false discovery analysis

```

python ~/pyseer/scripts/count_patterns.py
./pyseer_out/struct_patterns_AZM.txt

```

which gives

```

Patterns:      3239
Threshold:     1.54E-05

```

We can now summarise the rearrangement results using *R*

```

gono_struct_was <- do.call(rbind, map(antibiotics, function(ant){
  tbl <- fread(paste(c("./gono_harris_gwas/",ant,"_struct_was.txt"), collapse
= ""), data.table = FALSE)
  tbl$antibiotic <- ant
  return(tbl)
}))

gono_struct_was <- gono_struct_was[order(gono_struct_was$lrt-pvalue),]
gono_struct_was <- gono_struct_was[!grepl("bad-chisq",
gono_struct_was$notes),]

# threshold from running count_patterns in pyseer
sig_threshold <- 0.05/(3239*length(antibiotics))

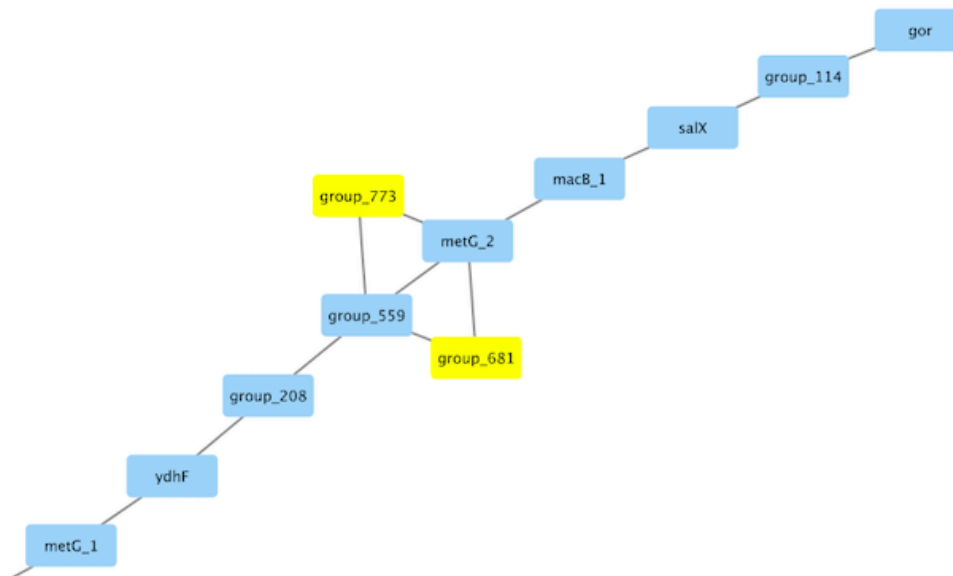
sum(gono_struct_was$lrt-pvalue<sig_threshold)
top_results <- gono_struct_was[gono_struct_was$lrt-pvalue<sig_threshold,]
top_results <- top_results[!duplicated(top_results$variant),]

write.csv(top_results,
file="./gono_harris_gwas/gono_struct_was_annotated.csv", quote = FALSE)

```

Inspecting the genome graph neighbourhood of a gene of interest

Looking at the genome graph in Cytoscape we can easily identify region in the genome graph that suggest some structural variation such as here:



We have colored nodes/genes in the graph that are supported by all samples in blue and genes that are supported by fewer samples in yellow. From this representation we cannot draw any conclusions about which paths through the genome graph are supported by which samples and/or how many different paths through that region of variation are there altogether. Here, we can use `panaroo-gene-neighbourhood`, which will find all the paths that run through one particular node in the graph.

```
panaroo-gene-neighbourhood --gene metG_2 --graph final_graph.gml --out  
neighbourhood.txt
```

Looking at the output in `neighbourhood.txt` we can see that there are three unique paths through these nodes. The dominant one excludes genes 'group_774' and 'group_681'.

NOTE: the direction of each path is not guaranteed.

support	members	path
46	14410_4#2,14410_6#47,14520_4#27,...	cpnA,metG_1,ydhF,group_208,group_559,metG_2,macB_1,salX,group_114,gor,bioY2
3	14520_7#67,15531_4#68,15531_4#73	bioY2,gor,group_114,salX,macB_1,metG_2,group_681,group_559,group_208,ydhF,metG_1
1	19646_1#31	bioY2,gor,group_114,salX,macB_1,metG_2,group_773,group_559,group_208,ydhF,metG_1

By default for each sample this will explore the five genes downstream and upstream from the target gene. We can add `--expand_no EXPAND NO` to change this behavior. For example running the same example as

```
panaroo-gene-neighbourhood --gene metG_2 --graph final_graph.gml --out
neighbourhood.txt --expand_no 2
```

gives

support	members	path
46	14410_4#2,14410_6#47,14520_4#27,...	group_208,group_559,metG_2,macB_1,salX
3	14520_7#67,15531_4#68,15531_4#73	salX,macB_1,metG_2,group_681,group_559
1	19646_1#31	salX,macB_1,metG_2,group_773,group_559

When we are interested in the exact paths through the graph it is sometime useful to turn off the edge filtering that Panaroo performs. This will allow us to see edges even if they look like they could be the result of misassemblies. This can be done by running Panaroo as

```
panaroo -i *.gff -o output_panaroo/ --clean-mode strict -t 24 --
no_clean_edges
```

Parameters

```
usage: panaroo-gene-neighbourhood [-h] --gene GENE [--genome_id GENOME_ID]
                                   --graph GRAPH [--expand_no EXPAND_NO]
                                   [--out OUT]
```

Explore gene neighbourhood

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>--gene GENE</code>	gene of interest
<code>--genome_id GENOME_ID</code>	genome ID of interest (default=ALL)
<code>--graph GRAPH</code>	genome graph gml ('final_graph.gml')
<code>--expand_no EXPAND_NO</code>	lengths of the path that will be expanded on in a radius the target gene (default=5)
<code>--out OUT</code>	output file

Output updated GFF files

After running Panaroo you can generate updated (simplified) GFF files for each isolate. These remove the filtered and add the refound gene annotations generated by Panaroo. Only CDS sequences considered by Panaroo are included in these simplified GFF files.

The GFF files can be generated by running the following command using the same output folder as specified when originally running Panaroo

```
panaroo-generate-gffs -i *.gff -o panaroo_out
```

The resulting GFF files can then be found in `./panaroo_out/panaroo_gffs/`

Parameters

```
usage: generate_gff_files [-h] -o OUTPUT_DIR -i INPUT_FILES [INPUT_FILES ...]
                        [-f {gff3,prokka}] [-t N_CPU] [--verbose]
```

Generate GFF annotation files for each isolate from the pan-genome graph

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>-t N_CPU, --threads N_CPU</code>	number of threads to use (default=1)
<code>--verbose</code>	print additional output

Input/output:

<code>-o OUTPUT_DIR, --out_dir OUTPUT_DIR</code>	location of the Panaroo output directory
<code>-i INPUT_FILES [INPUT_FILES ...], --input INPUT_FILES [INPUT_FILES ...]</code>	input GFF3 files used to run panaroo. These are required Can also take a file listing each gff file line by line.

GFFs:

<code>-f {gff3,prokka}, --f {gff3,prokka}</code>	Output format for the GFF files. 'prokka' with included FASTA, or strict 'gff3'. Default: 'prokka'
--	--

Estimating the Pangenome Size

Panstripe

We recommend using the R package [panstripe](#) to investigate pangenome dynamics and size. The package can produce common summary plots and allows for the comparison of gene gain and loss rates between lineages and datasets.

Alternative models

Panaroo also includes a number of alternative scripts for estimating the pangenome size as well as gene gain and loss rates according to both the Finitely Many Genes (FMG) model and the Infinitely Many Genes (IMG) model. These models are described in a number of papers listed in the references at the bottom of this page.

These model based approaches are preferable to the common accumulation curves often used in pangenome analyses. Such curves fail to account for the phylogenetic relationship between isolates and thus it is inappropriate to use them to compare different data sets.

Infinitely Many Genes model

The IMG model allows for gene gain from an unbounded reservoir of new genes, and gene loss. As the reservoir is unbounded, the same gene gain event can only occur once. This might represent horizontal gene transfer from a diverged taxa with gene loss representing the conversion of genes to pseudogenes or deletion in reproduction. This model is described in Baumdicker et al. 2012 and Collins et al. 2012.

To estimate the parameters of this model, a dated phylogeny based on the core genome is required. Such phylogenies can be produced using [BEAST](#) or by combining faster methods such as [IQ-TREE](#) and [BactDating](#)

An implementation following that in Collins et al. 2012 is given in Panaroo and can be run as

```
mkdir img_results
panaroo-img --pa gene_presence_absence.Rtab -o img_results -tree
dated_phylogeny.newick
```

Parameters

For a full description of the parameters and model please see the Collins et al. 2012 paper.

Estimate model parameters for either the Infinitely Many Genes Model using gene frequencies

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>--tree TREE</code>	A dated phylogeny.
<code>--pa PRESENCE_ABSENCE</code>	A presence/absence produced by Panaroo.
<code>-o OUTPUT_DIR, --out_dir OUTPUT_DIR</code>	location of an output directory
<code>-D {1,2}</code>	Number of separate rate classes to use for the dispensable genome. Can be either 1 or 2.
<code>--no_essential</code>	Removes essential gene class from model
<code>--no_constraint</code>	Removes constraint that u/v must equal the genome size.
<code>--model {coalescent,fixed}</code>	Model to fit. Can be one of 'coalescent' or 'fixed'.
<code>--fit {cp,gf}</code>	Whether to use the gene frequency spectrum or the core/pangenome size for fitting (default=gf)
<code>--init_u1 U1</code>	Initial value for u1 (default=0.01).
<code>--init_u2 U2</code>	Initial value for u2 (default=0.01).
<code>--init_v1 V1</code>	Initial value for v1 (default=0.01).
<code>--init_v2 V2</code>	Initial value for v2 (default=0.01).
<code>--init_ess GESS</code>	Initial value for the number of essential genes (default=2000).
<code>--verbose</code>	print additional output
<code>--version</code>	show program's version number and exit

Finitely Many Genes model

The FMG model differs from the IMG mode in that the same gene can be gained and lost multiple times. Moreover, the pool of genes from which a new gene can be drawn is finite. The FMG model is a version of the standard two-state phylogenetic models which are often used to study the evolution of binary traits.

We have implemented a simplified version of that described in Zamani-Dahaj et al. 2016.

Similar to the IMG model a dated phylogeny based on the core genome is required. The script can then be run as

```
panaroo-fmg --tree dated_phylogeny.newick --pa
gene_presence_absence_renamed.Rtab -o fmg_results.txt
```

Parameters

For a full description of the parameters and model please see the Zamani-Dahaj et al. 2016 paper.

Estimate model parameters for the Finitely Many Genes Model

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>--tree TREE</code>	A dated phylogeny.
<code>--pa PRESENCE_ABSENCE</code>	A presence/absence produced by Panaroo.
<code>-o OUTPUTFILE, --outfile OUTPUTFILE</code>	Name of outputfile.
<code>--nboot NB00T</code>	The number of sub-sampling bootstrap iterations to perform.
<code>-t N_CPU, --threads N_CPU</code>	number of threads to use (default=1)
<code>--verbose</code>	print additional output
<code>--version</code>	show program's version number and exit

Alternative Tools

The output from Panaroo can also be used as input to a number of other post-processing tools for estimating gene gain and loss rates. These include

- [PanX](#)
- [panicmage](#)
- [pantagruel](#)

References

Collins,R.E. and Higgs,P.G. (2012) Testing the Infinitely Many Genes Model for the Evolution of the Bacterial Core Genome and Pangenome. *Mol. Biol. Evol.*, 29, 3413–3425.

Baumdicker,F., Hess,W.R. and Pfaffelhuber,P. (2012) The infinitely many genes model for the distributed genome of bacteria. *Genome Biol. Evol.*, 4, 443–456.

Zamani-Dahaj,S.A., Okasha,M., Kosakowski,J. and Higgs,P.G. (2016) Estimating the Frequency of Horizontal Gene Transfer Using Phylogenetic Models of Gene Gain and Loss. *Mol. Biol. Evol.*, 33, 1843–1857.

Co-Evolution and Epistasis Analysis

Panaroo includes an implementation of the [Spydrpick](#) algorithm for identifying co-evolving genes and genes involved in epistatic interactions. The Spydrpick algorithm makes use of pairwise Mutual Information (MI) values to identify interactions that appear to be significantly different from the background distribution.

The Spydrpick algorithm requires either a phylogeny or clustering to control for population structure. A phylogeny can be built from the core genome using [lqtree](#). Suitable clusterings can be generated for the pangenome using [Poppunk](#) or from the core genome using [fastbaps](#).

The spydrpick algorithm can be run using

```
mkdir coevo_results
panaroo-spydrpick -i gene_presence_absence.Rtab -o coevo_results --clusters
clusters.csv
```

Parameters

For a full description of the algorithm and its parameters please see the Spydrpick publication.

Identify associations between genes using the Spydrpick algorithm

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>--quantile QUANTILE</code>	the quantile used to determine a threshold for keeping
	MI values (default=0.9).
<code>--version</code>	show program's version number and exit

Input/output:

<code>-i PA_FILE, --input PA_FILE</code>	gene presence absence file (.Rtab) generated by Panaroo
<code>-o OUTPUT_DIR, --out_dir OUTPUT_DIR</code>	location of an output directory
<code>--tree TREE_FILE</code>	phylogeny in newick format for weighting samples to control for population structure
<code>--clusters CLUSTER_FILE</code>	sample clusters for weighting to control for population structure. format: 'sample_name,cluster_id'

Output

The program will produce a file `gene_pa_spydrpick.csv` which includes those gene pairs with MI values above the specified threshold. It includes an 'outlier' column with values

Outlier Flag	Description
0	non-outlier
1	outlier
2	extreme outlier

Alternative Tools

The output from Panaroo can also be used as input to a number of other co-evolution estimation tools. These include

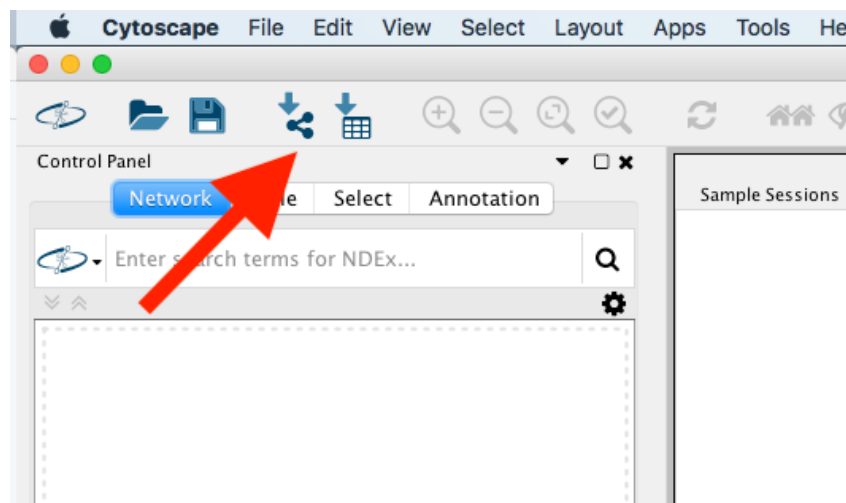
- [pantagruel](#)
- [coinfinder](#)

Visualising the Pangenome in Cytoscape

Loading

Panaroo produces a GML file (`final_graph.gml`) for input to [Cytoscape](#).

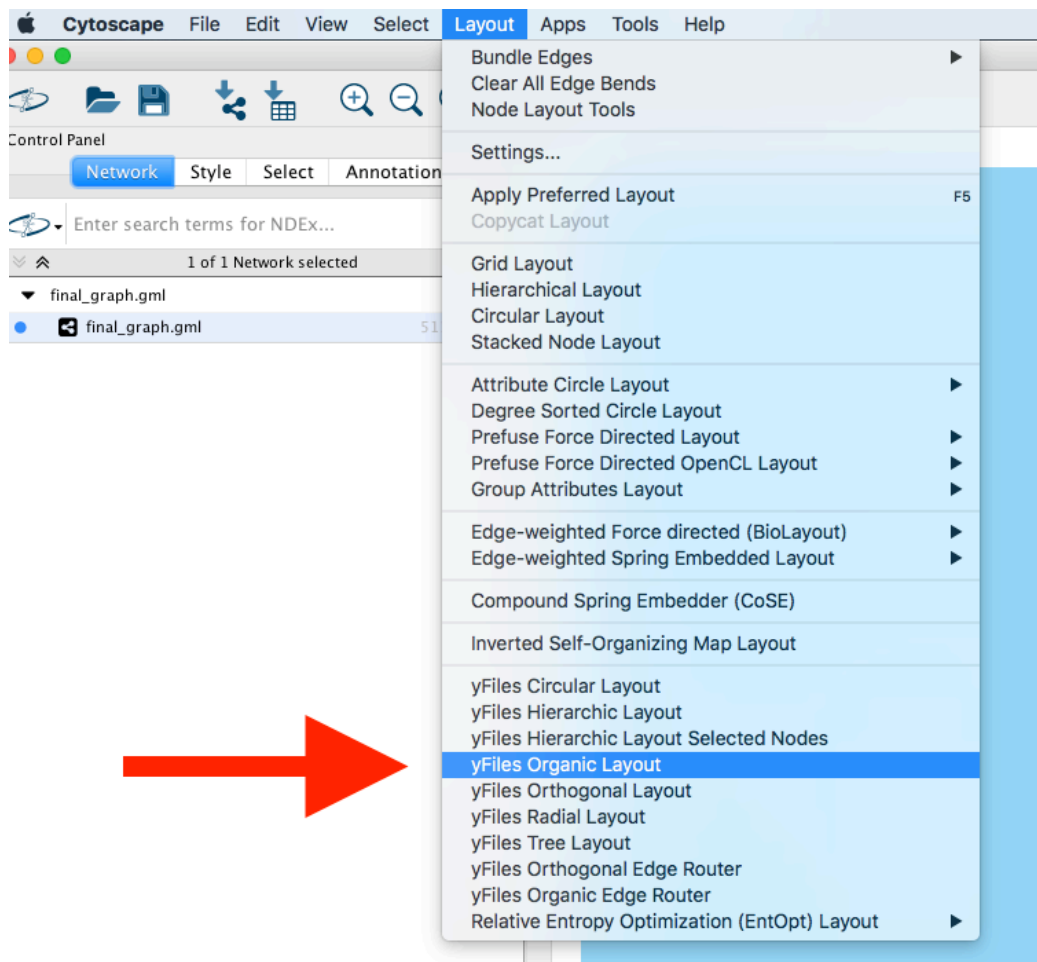
This graph can be loaded by clicking on the 'Import Network' button at the top right of the Cytoscape application and navigating to the `final_graph.gml` file. We have indicated this in the figure below.



Layout

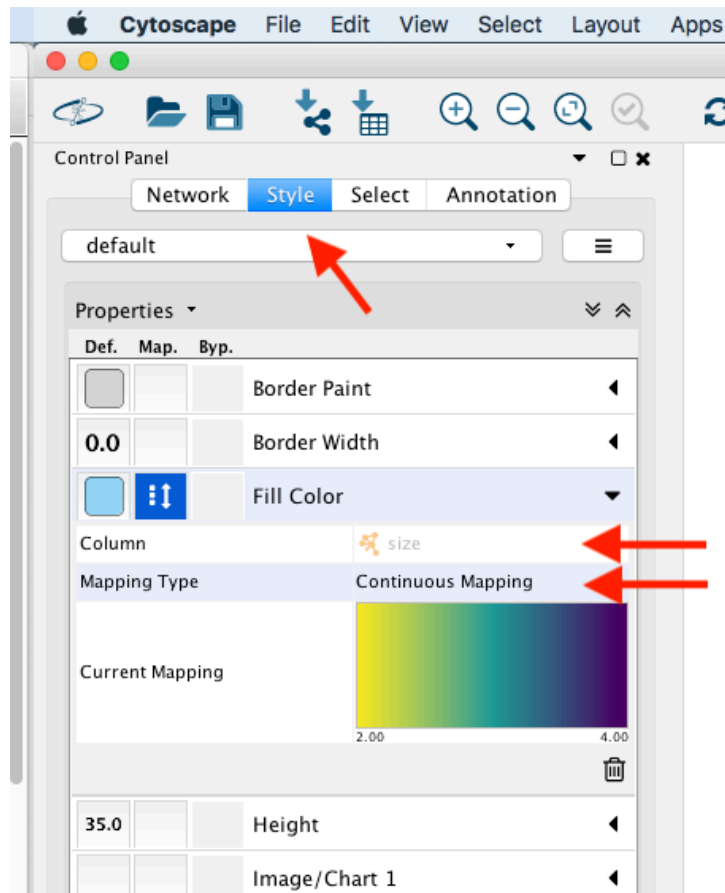
We recommend the [yFiles Organic Layout](#) algorithm to arrange the nodes of the graph for visualisation.

Once the [yFiles](#) plugin has been installed this layout can be called by selecting it from the dropdown menu in Cytoscape. This is indicated below.

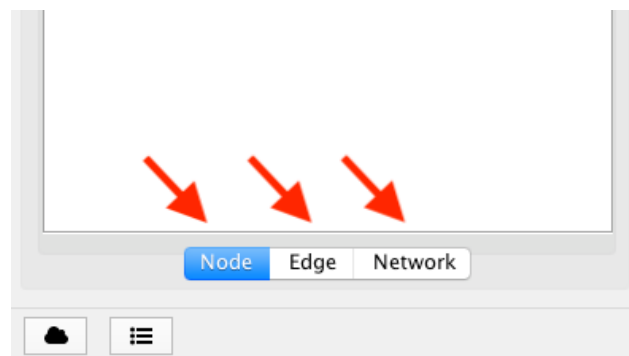


Annotation

The style of the nodes and edges in the graph can be used to convey extra information. These attributes can be controlled using the 'style' tab in cytoscape. Below we indicate how to colour the nodes based on the number of genomes that they contain. This helps to visualise the core and accessory genome based upon the colour of each node.



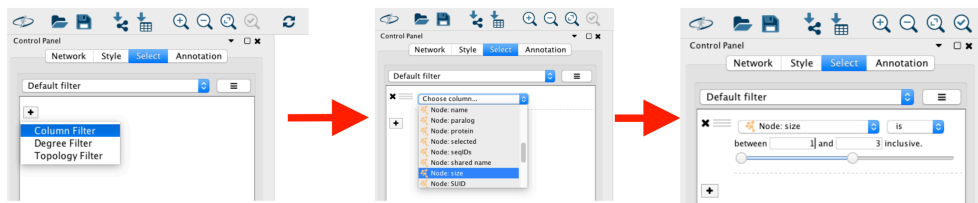
The style tabs for edges, nodes and the network can be toggled at the bottom of the style tab as indicated below.



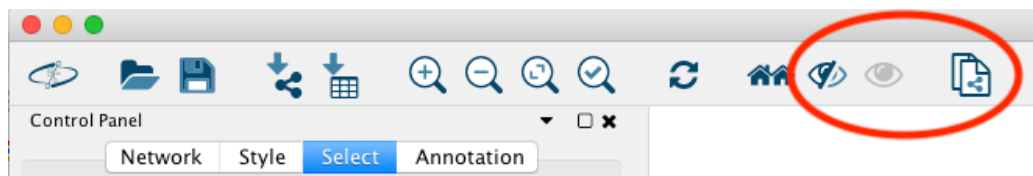
Filtering

Sometimes it is useful to focus on a subset of the graph. Nodes can be selected using the mouse or rules in the 'select' tab. These can then either be converted into a subgraph for further analysis or filtered out from the main graph.

The figure below indicates how to select nodes using a set of rules. Here we are selecting all nodes that appear in between 1 and 3 genomes.



The buttons in the figure below can then be used to either create a new graph from the selected nodes or to filter them out.

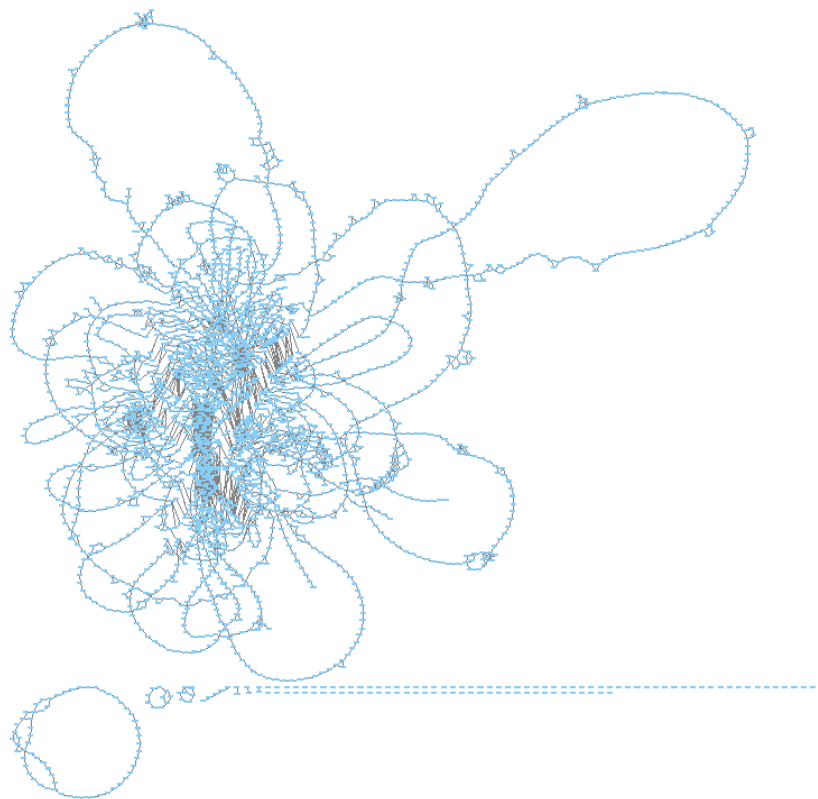


Additional Metadata

Additional metadata can be imported to cytoscape using the "Import table from file" button, next to the "Import network" button. These data can be associated with a graph and then user for additional filtering or annotation. The name attribute in the pangenome graph is a good option to index the imported spreadsheet for associating additional gene data with the pangenome graph. See [Meta-data Annotation](#) for additional details.

Using a reference genome to produce less convoluted layouts

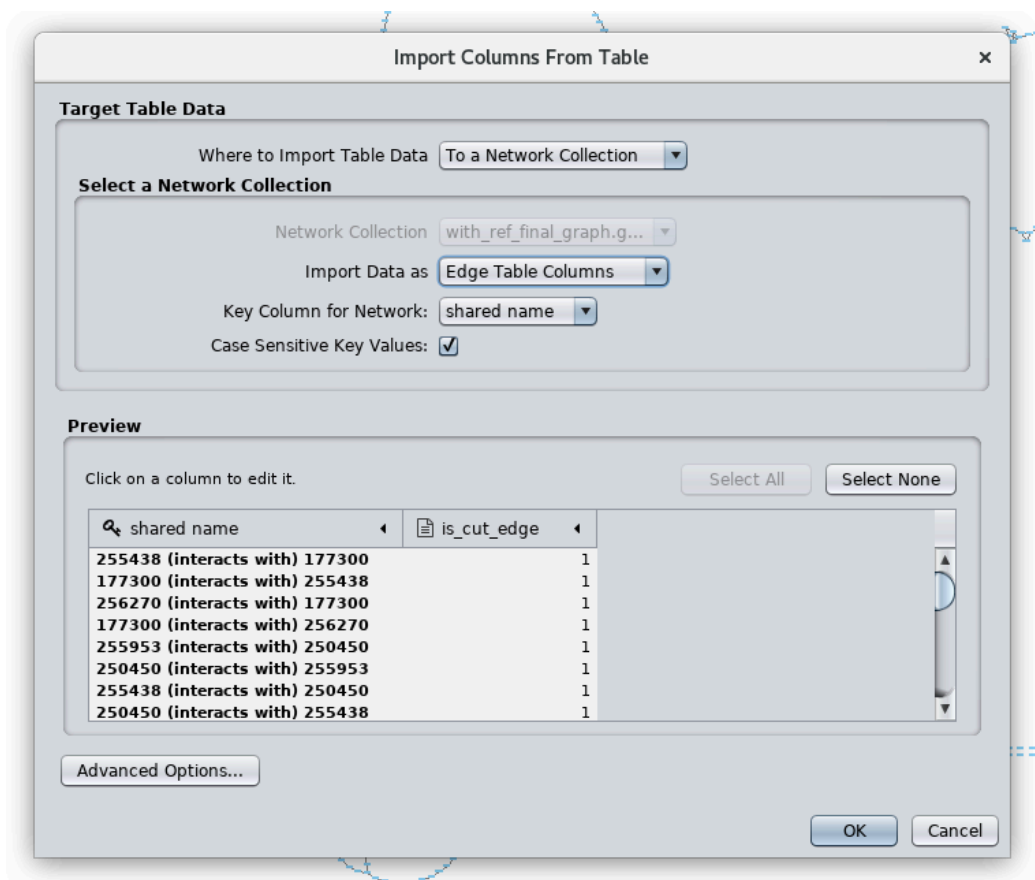
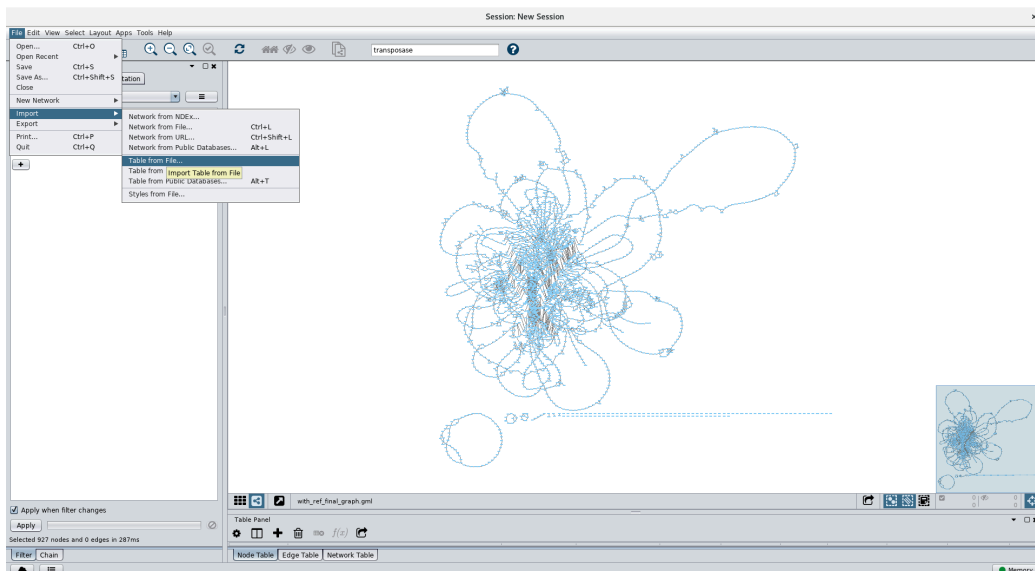
The standard organic layout algorithm in Cytoscape is very effective at laying out very big networks but still produces some times very convoluted networks.



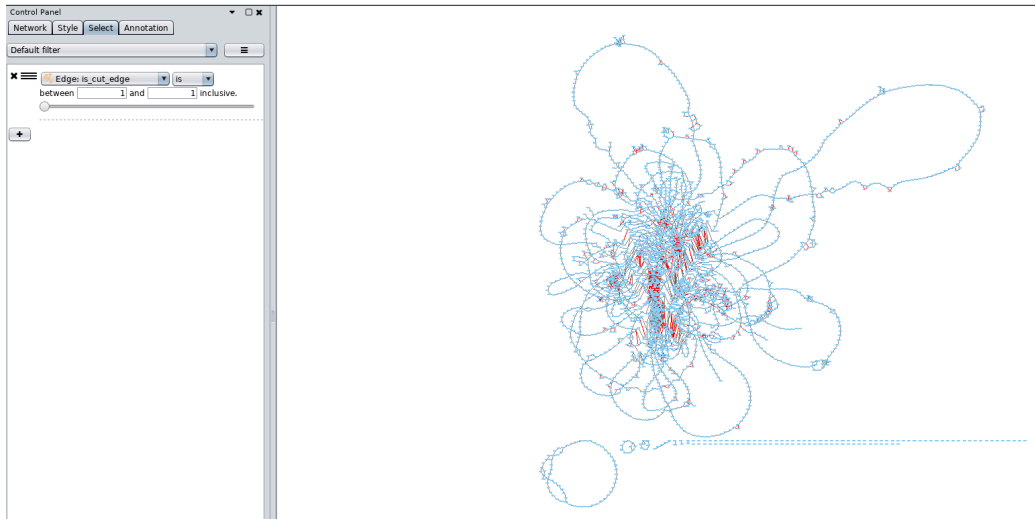
If we include a completely sequenced genome such as most reference genomes we can use this as a backbone for a simpler layout. Specifically, we identify paths through the genome graph that connect regions in the reference genome that are far apart on the chromosome. We call them long-range connections. We have implemented an algorithm based on a minimum capacity graph-cut to identify any such path and finally a set of edges that disconnects those long-range connections in the genome graph.

```
python ~/repos/panaroo/scripts/reference_based_layout.py 0 final_graph.gml  
capacity_cut_edges.txt --add_reference_edges
```

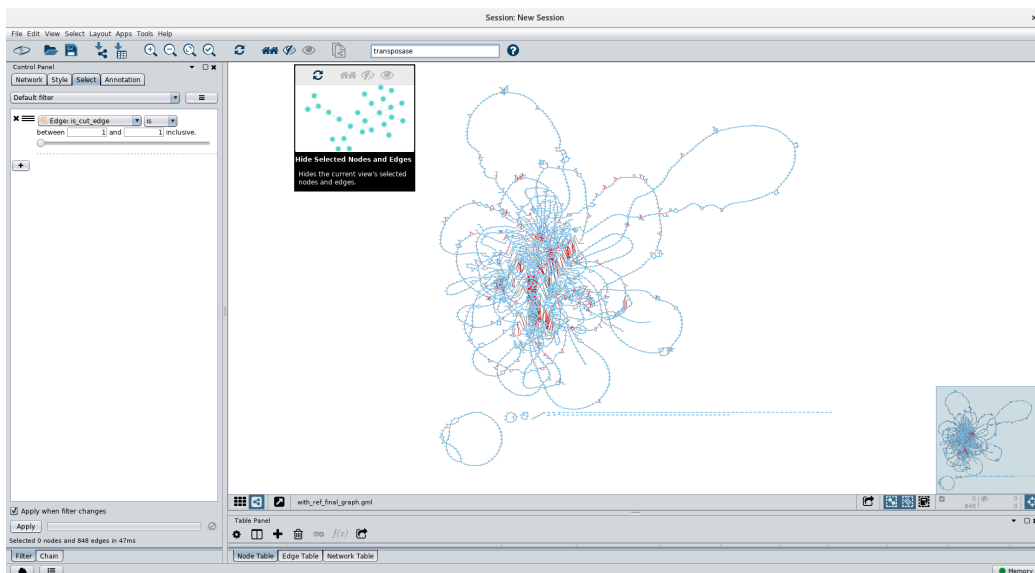
As input this requires the id of the reference genome in the Panaroo input and the graph. Since Panaroo sometimes removes links between consecutive genes in the reference genome, we currently advise to run this script with the `--add_reference_edges` flag. The script will generate a version of the genome graph `final_graph_with_ref.gml` that is guaranteed to keep all connection in the reference genome. As main output of the layout script we get a table with all edges that break up long-range connection `capacity_cut_edges.txt`. Next we need to import these edges into Cytoscape:



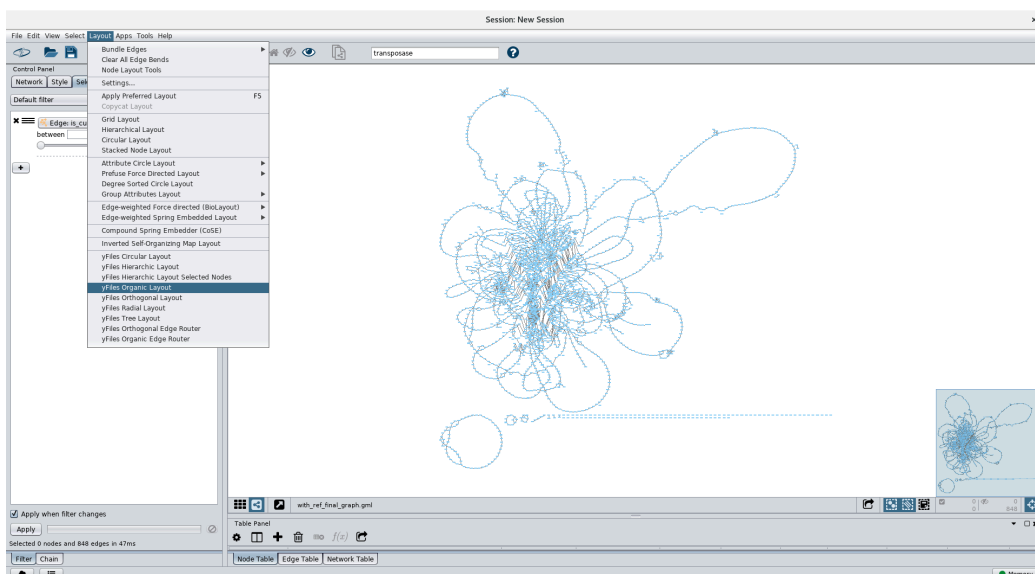
Make sure that you select "Import Data as: Edge Table columns" Next in the Control Panel go to Select and a filter for Edge: is_cut_edge. I will highlight all edges that we have just imported.



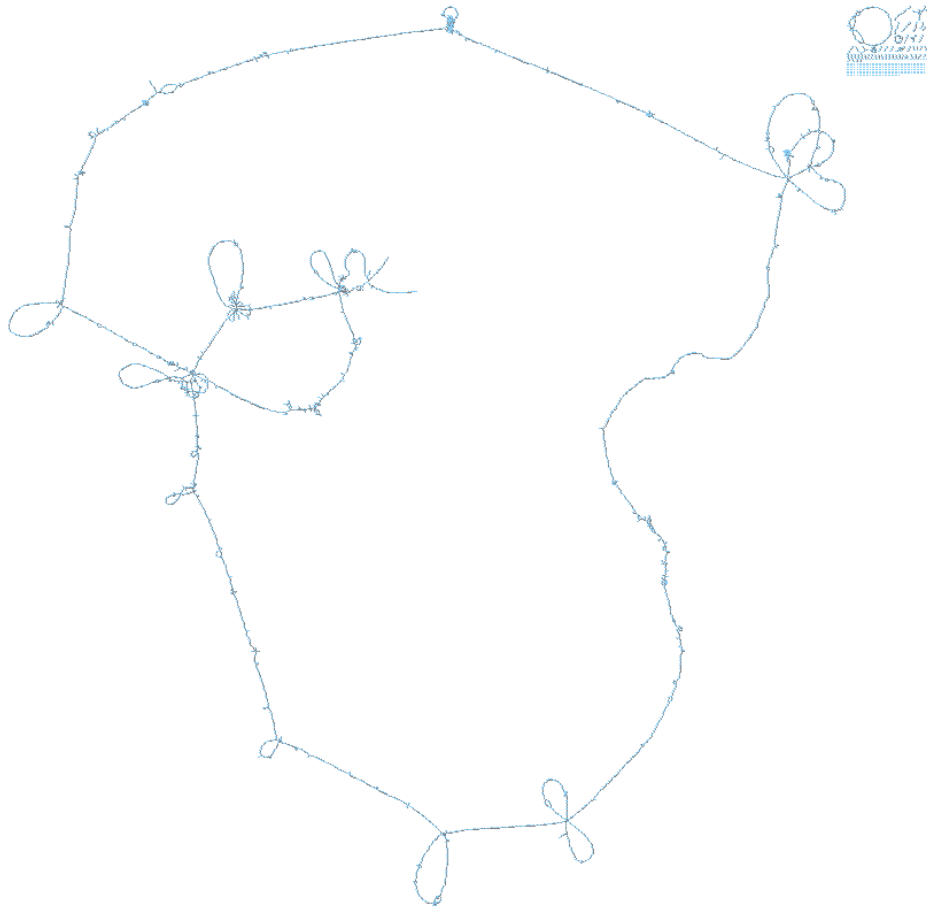
Then hide all those edges:



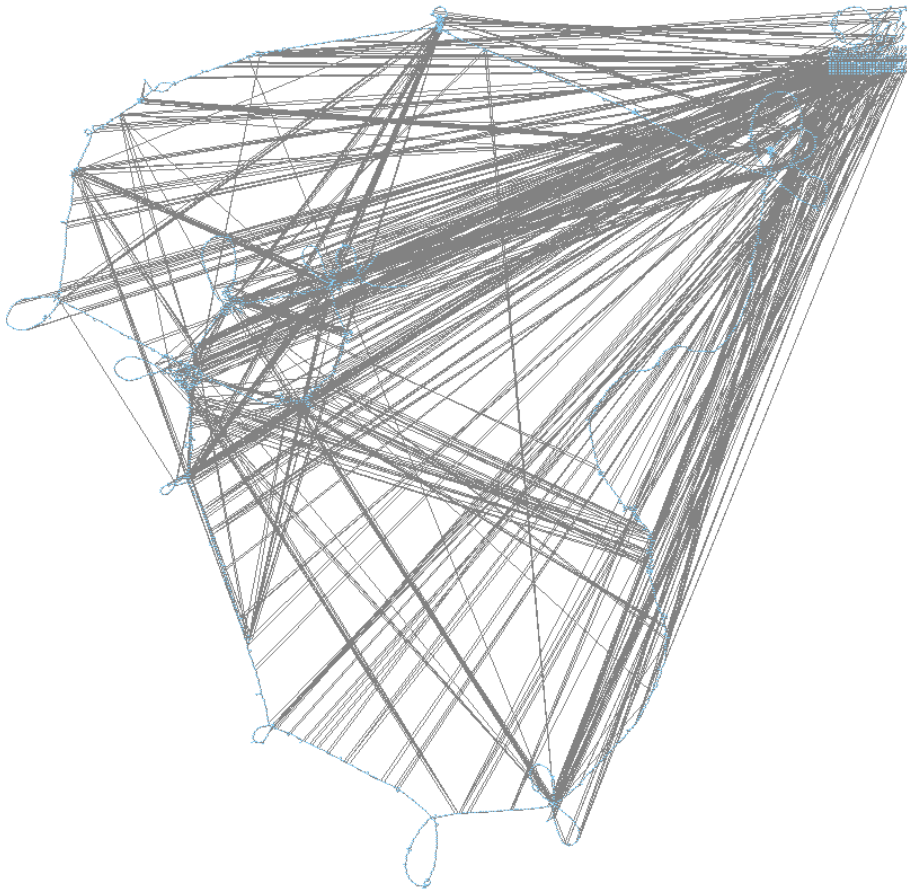
Finally apply the yFiles organic layout to re-lay out the genome graph



The resulting layout:



We can click on Show All Nodes and Edges to make all long-range connections re-appear.

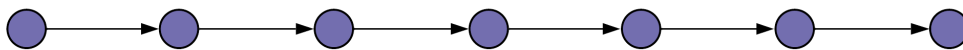
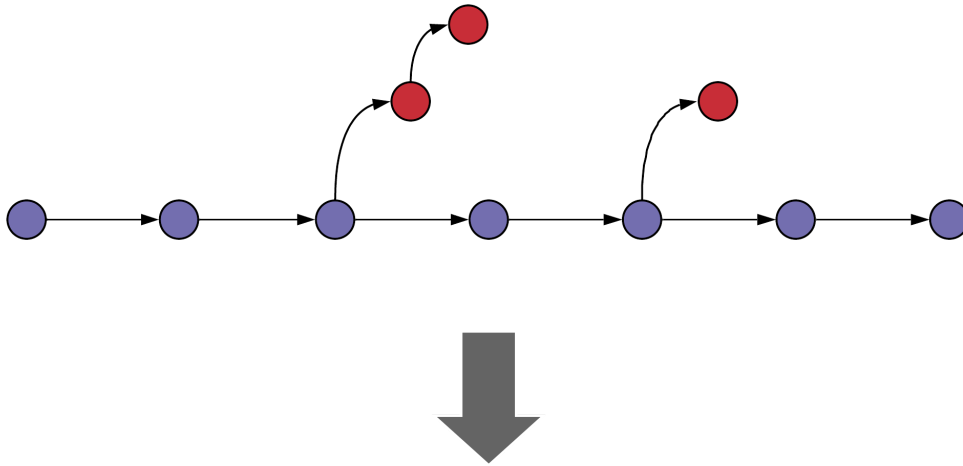


Meta-data annotation e.g. sequence types

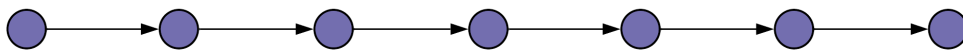
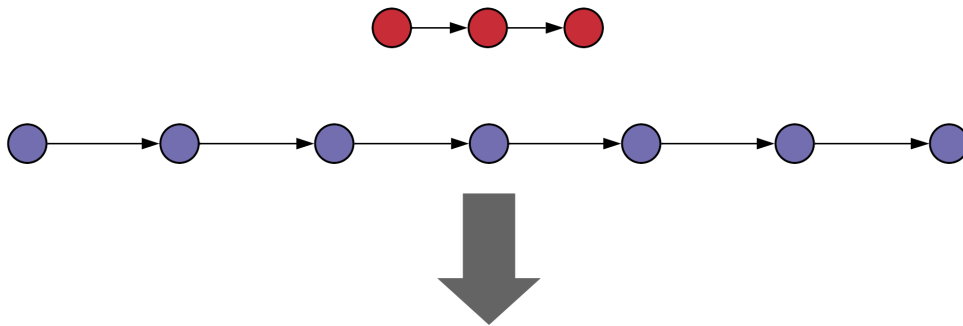
Page still under construction. Please see [Visualising the Pangenome](#) for general instructions on how to filter and annotate the pangenome graph in cytoscape.

Graph Correction

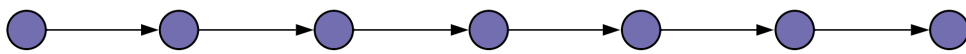
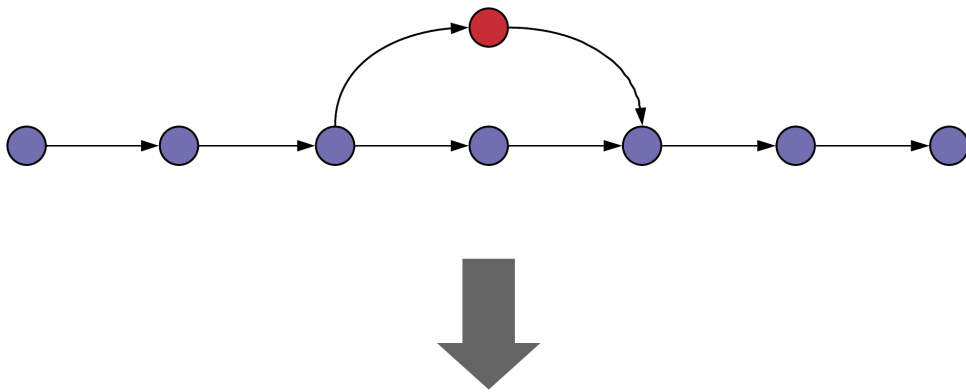
Spurious Ends



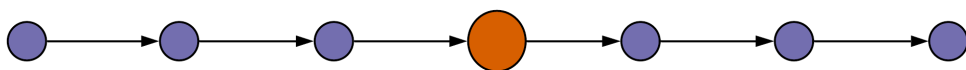
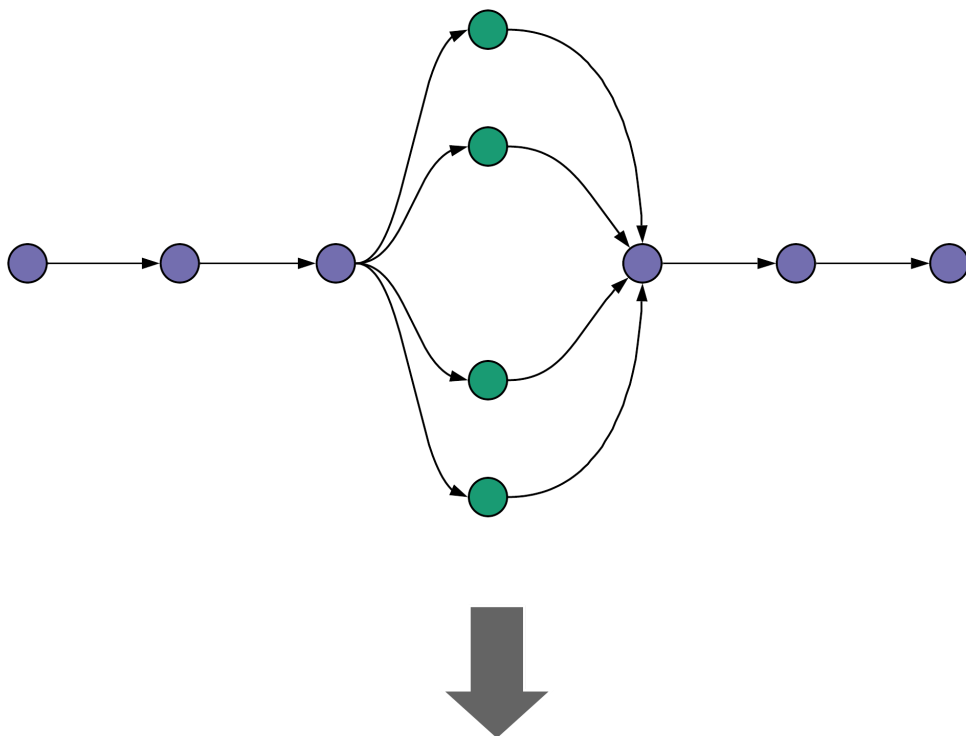
Contamination Removal



Mistranslations



Gene Families



Re-finding Genes

